

Fast Fourier Transform & Linear Programming

Polynomial Multiplication via FFT · Simplex & Duality

Algorithm Engineering

CLRS Chapters 30 & 29

Today's Roadmap

1 FFT and Polynomial Multiplication

2 Linear Programming

Study Goal

By the end, you should be able to explain the main idea, write the core formulas/algorithms, derive the running times, and solve standard exam-style questions on FFT and LP duality.

Part I — Fast Fourier Transform

Chapter 30: Fast Multiplication of Polynomials

1. The Motivation: Polynomial Multiplication

Problem

Given two polynomials $A(x)$ and $B(x)$, compute $C(x) = A(x)B(x)$ efficiently.

Suppose $A(x) = \sum_{j=0}^{n-1} a_j x^j$ and $B(x) = \sum_{j=0}^{n-1} b_j x^j$. Then $C = AB$ is the coefficient convolution, with each c_r formed from products $\sum_{j=0}^r a_j b_{r-j}$.

- There are $\Theta(n)$ output coefficients.
- The straightforward method performs $\Theta(n^2)$ coefficient products.

Goal

Reduce $\Theta(n^2)$ to $\Theta(n \log n)$.

2. Two Representations of a Polynomial

Coefficient Representation

A degree-bound n polynomial is stored as

$$a = (a_0, a_1, \dots, a_{n-1}), \quad A(x) = \sum_{j=0}^{n-1} a_j x^j.$$

- Convenient for addition: $\Theta(n)$.
- Evaluate one point using Horner's rule: $\Theta(n)$.
- Multiplication: $\Theta(n^2)$.

Point-Value Representation

Store n distinct points and their values:

$$\{(x_0, A(x_0)), \dots, (x_{n-1}, A(x_{n-1}))\}.$$

- Addition: add values at the same points.
- Multiplication: multiply values at the same points.
- Pointwise multiplication costs only $\Theta(n)$.

Where the Speedup Will Come From

Point-value form makes multiplication easy. The challenge is converting coefficient $\leftrightarrow$ point-value efficiently.

3. Why Do We Need $2n$ Evaluation Points?

If A and B each have degree-bound n , then

$$\deg(AB) \leq (n - 1) + (n - 1) = 2n - 2,$$

so $C = AB$ has degree-bound $2n$.

Interpolation Requirement

A degree-bound m polynomial is uniquely determined by m distinct point-value pairs. Therefore, to recover C we need $2n$ distinct points.

- Pad A and B with n high-order zeros.
- Evaluate both padded polynomials at the same $2n$ points.
- Multiply pointwise to obtain values of C .
- Interpolate C from the $2n$ values.

The bottleneck

Arbitrary-point evaluation/interpolation is still $\Theta(n^2)$. CLRS chooses a special set of points: complex roots of unity.

4. Uniqueness of Interpolation (Theorem 30.1)

Statement

Given n point-value pairs with distinct $x_0, \dots, x_{n-1}$, there is a unique polynomial of degree-bound n passing through all the pairs.

- The equations are

$$y_k = \sum_{j=0}^{n-1} a_j x_k^j.$$

- In matrix form: $Va = y$, where V is a Vandermonde matrix.
- Because the x_k are distinct, the Vandermonde matrix is invertible.

What We Use Today

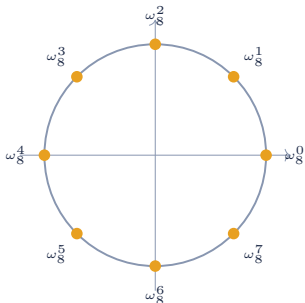
We only need the theorem's conclusion: evaluation and interpolation are inverse, well-defined operations. The Vandermonde inverse proof is omitted.

5. Roots of Unity: The Special Evaluation Points

Definition

A complex n th root of unity is a complex number ω satisfying $\omega^n = 1$. The n roots are

$$\omega_n^k = e^{2\pi i k/n}, \quad k = 0, 1, \dots, n-1, \quad \omega_n = e^{2\pi i/n}.$$



- All roots lie on the unit circle.
- They are equally spaced by angle $2\pi/n$.
- Every root is a power of the principal root ω_n .
- Multiplication corresponds to adding exponents modulo n .

$$\omega_n^j \omega_n^k = \omega_n^{(j+k) \bmod n}.$$

CLRS Convention

This chapter uses $e^{+2\pi i/n}$.

Some signal-processing texts use the conjugate convention.

6. Cancellation Lemma and -1

For $n > 0$, $k \geq 0$, and $d > 0$,

$$\omega_{dn}^{dk} = \omega_n^k.$$

Proof: $\omega_{dn}^{dk} = e^{2\pi i(dk)/(dn)} = e^{2\pi i k/n} = \omega_n^k.$

If n is even, then

$$\omega_n^{n/2} = \omega_2 = -1.$$

Why This Matters for FFT

The roots are related across different problem sizes, and the second half of the roots will differ by a minus sign.

7. Halving Lemma — The Main Structural Trick

If n is even, then squaring the n roots of unity gives exactly the $n/2$ roots of unity, with every $n/2$ -th root appearing twice.

For $k = 0, \dots, n-1$,

$$(\omega_n^k)^2 = \omega_n^{2k} = \omega_{n/2}^k \quad \text{by the cancellation lemma.}$$

Moreover,

$$(\omega_n^{k+n/2})^2 = (\omega_n^k)^2.$$

The FFT Insight

An n -point evaluation problem becomes two $(n/2)$ -point evaluation problems. This is the source of the divide-and-conquer speedup.

8. Summation Lemma — Why Orthogonality Appears

If k is not divisible by n , then

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = 0.$$

Let $r = \omega_n^k$. Since $r \neq 1$,

$$\sum_{j=0}^{n-1} r^j = \frac{r^n - 1}{r - 1} = 0, \quad r^n = (\omega_n^k)^n = 1.$$

- The complex roots cancel around the unit circle.
- This lemma is used directly in the inverse DFT proof.
- It also explains the “orthogonality” behind Fourier transforms.

9. Discrete Fourier Transform (DFT)

Definition

For $A(x) = \sum_{j=0}^{n-1} a_j x^j$, define

$$y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j \omega_n^{kj}, \quad k = 0, \dots, n-1.$$

The vector y is the DFT of a :

$$y = \text{DFT}_n(a).$$

- Input: coefficient vector a .
- Output: values of the polynomial at all n roots of unity.
- Direct computation: n outputs $\times n$ terms $= \Theta(n^2)$.

FFT = fast way to compute the same DFT

The FFT changes only the computation strategy; it does not change the DFT being computed.

10. DFT as a Matrix-Vector Product

$$y = V_n a, \quad (V_n)_{k,j} = \omega_n^{kj}.$$
$$\begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \cdots & \omega_n^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \cdots & \omega_n^{(n-1)^2} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix}.$$

Important distinction

The DFT is a mathematical transformation. The FFT is an algorithm for computing it faster.

11. FFT: Split Even and Odd Coefficients

Define

$$A^{\text{even}}(x) = a_0 + a_2x + \cdots + a_{n-2}x^{n/2-1},$$
$$A^{\text{odd}}(x) = a_1 + a_3x + \cdots + a_{n-1}x^{n/2-1}.$$

Then

$$A(x) = A^{\text{even}}(x^2) + xA^{\text{odd}}(x^2).$$

Evaluate at $x = \omega_n^k$:

$$A(\omega_n^k) = A^{\text{even}}((\omega_n^k)^2) + \omega_n^k A^{\text{odd}}((\omega_n^k)^2).$$

Apply the Halving Lemma

$(\omega_n^k)^2$ runs through only the $n/2$ roots of unity of order $n/2$ (each twice). Thus each half-polynomial is evaluated by a size- $n/2$ DFT.

12. FFT Combine Step and Twiddle Factors

Let

$$y_k^{\text{even}} = A^{\text{even}}(\omega_n^{2k}), \quad y_k^{\text{odd}} = A^{\text{odd}}(\omega_n^{2k}).$$

For $0 \leq k < n/2$,

$$y_k = y_k^{\text{even}} + \omega_n^k y_k^{\text{odd}},$$

$$y_{k+n/2} = y_k^{\text{even}} - \omega_n^k y_k^{\text{odd}}.$$

Why the Minus Sign?

$$\omega_n^{k+n/2} = \omega_n^k \omega_n^{n/2} = -\omega_n^k.$$

So one multiplication by ω_n^k is reused with opposite signs.

The factors ω_n^k are called **twiddle factors** in CLRS.

13. FFT Recurrence and Running Time

$$T(n) = 2T(n/2) + \Theta(n).$$

The two recursive calls cost $2T(n/2)$; the combine step costs $\Theta(n)$.

- At each recursion level, the total combine work is $\Theta(n)$.
- There are $\log_2 n$ levels.

$$T(n) = \Theta(n \log n)$$

Exam pattern

Always explain both parts: *why the subproblems are half-sized* (halving lemma) and *why combine is linear* (the $n/2$ twiddle-factor pairs).

14. Recursive-FFT Pseudocode

Algorithm: FFT(a, n)

```
1: if  $n = 1$  then
2:   return  $a$ 
3: end if
4:  $\omega_n \leftarrow e^{2\pi i/n}$ ;  $\omega \leftarrow 1$ 
5:  $a^{\text{even}} \leftarrow (a_0, a_2, \dots, a_{n-2})$ 
6:  $a^{\text{odd}} \leftarrow (a_1, a_3, \dots, a_{n-1})$ 
7:  $y^{\text{even}} \leftarrow \text{FFT}(a^{\text{even}}, n/2)$ 
8:  $y^{\text{odd}} \leftarrow \text{FFT}(a^{\text{odd}}, n/2)$ 
9: for  $k = 0$  to  $n/2 - 1$  do
10:   $t \leftarrow \omega y_k^{\text{odd}}$ 
11:   $y_k \leftarrow y_k^{\text{even}} + t$ 
12:   $y_{k+n/2} \leftarrow y_k^{\text{even}} - t$ 
13:   $\omega \leftarrow \omega \omega_n$ 
14: end for
15: return  $y$ 
```

Assumption in CLRS: n is an exact power of 2; otherwise pad with zeros when using this version.

15. Worked Example: 4-Point DFT

Let $a = (1, 2, 3, 4)$ and $\omega_4 = i$.

$$y_0 = 1 + 2 + 3 + 4 = 10,$$

$$y_1 = 1 + 2i + 3(-1) + 4(-i) = -2 - 2i,$$

$$y_2 = 1 - 2 + 3 - 4 = -2,$$

$$y_3 = 1 - 2i - 3 + 4i = -2 + 2i.$$

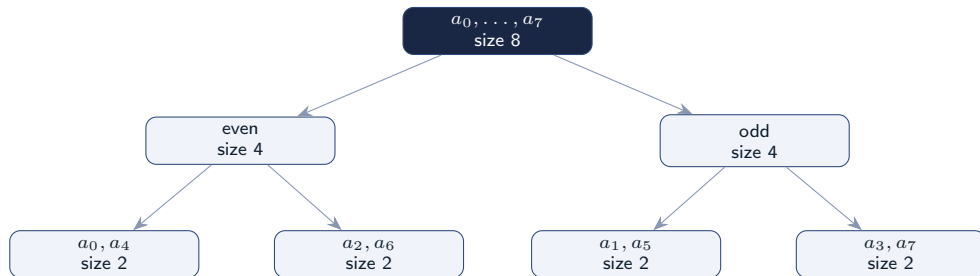
What the FFT Would Do

Split $(1, 2, 3, 4)$ into even coefficients $(1, 3)$ and odd coefficients $(2, 4)$, compute two 2-point DFTs, then combine them using 1 and i .

Sanity check

y_3 is the complex conjugate of y_1 because the input coefficients are real.

16. FFT Recursion Tree for $n = 8$



repeat until size 1

- Every node recursively splits into even and odd indexed coefficients.
- Height = $\log_2 8 = 3$.
- Total combine work per level is linear in the number of elements at that level.

17. Why the FFT Is Correct — Intuition, Not a Full Invariant Proof

Correctness Story

Every output y_k must equal $A(\omega_n^k)$.

- 1 The even/odd identity rewrites the original polynomial exactly.
- 2 The halving lemma says the squared evaluation points are precisely the smaller roots of unity.
- 3 Therefore the recursive calls compute the two terms at the correct smaller points.
- 4 The combine formulas reconstruct both halves of the n outputs; the cancellation lemma gives the required roots, and Corollary 30.4 supplies the minus sign.

Exam expectation

Be able to explain this chain of ideas and derive the two combine equations. A full recursive invariant proof is intentionally omitted.

18. Inverse DFT / Inverse FFT

Theorem 30.7 — Inverse DFT

The inverse DFT is

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k \omega_n^{-kj}, \quad j = 0, \dots, n-1.$$

To compute it, use the same FFT structure with:

- ω_n replaced by ω_n^{-1} ;
- the roles of input/output reversed;
- divide every final entry by n .

Running Time

$$\text{DFT}_n^{-1} = \Theta(n \log n).$$

CLRS proves the inverse-matrix formula using the summation lemma; the full matrix proof is omitted here.

19. Convolution Theorem

Theorem 30.8

For length- n vectors a, b (zero-padded to length $2n$),

$$a \otimes b = \text{DFT}_{2n}^{-1} \left(\text{DFT}_{2n}(a) \odot \text{DFT}_{2n}(b) \right),$$

where $\odot$ means componentwise multiplication.

Interpretation

Convolution in coefficient space $\longleftrightarrow$ pointwise multiplication in Fourier space.

This is the algebraic reason FFT is useful for both polynomial multiplication and many signal-processing tasks.

20. Complete FFT-Based Polynomial Multiplication

Four Steps

- 1 **Pad:** extend A and B to length $2n$ by adding high-order zero coefficients.
- 2 **Evaluate:** compute $\text{DFT}_{2n}(A)$ and $\text{DFT}_{2n}(B)$ with FFT.
- 3 **Multiply pointwise:** $C(\omega_{2n}^k) = A(\omega_{2n}^k)B(\omega_{2n}^k)$.
- 4 **Interpolate:** apply inverse FFT to recover the coefficients of C .

Total Cost

$$\Theta(n) + \Theta(n \log n) + \Theta(n) + \Theta(n \log n) = \Theta(n \log n).$$

The Reformulation Pattern

Convert to a representation where the difficult operation becomes easy, perform the easy operation, then convert back.

FFT: What You Should Know for an Exam

- Define coefficient and point-value representations.
- Explain why pointwise multiplication is linear time and why $2n$ points are needed.
- Define ω_n and state the cancellation, halving, and summation lemmas.
- Derive

$$A(x) = A^{\text{even}}(x^2) + xA^{\text{odd}}(x^2).$$

- Derive the butterfly/combine formulas and the recurrence $T(n) = 2T(n/2) + \Theta(n)$.
- State the inverse DFT formula and explain the ω_n^{-1} and $1/n$ changes.
- Describe the four-step polynomial multiplication pipeline and its $\Theta(n \log n)$ cost.

Not required here

Vandermonde-inverse proof, full recursive invariant proof, FFT circuit/bit-reversal implementation, and numerical round-off analysis.

Part II — Linear Programming

Chapter 29: Linear Programming, Simplex, and Duality

21. What Is a Linear Program?

Definition

A linear program (LP) optimizes a linear objective function subject to a finite set of linear constraints. CLRS uses the following standard form for a maximization LP:

$$\max c^T x \quad \text{s.t.} \quad Ax \leq b, x \geq 0.$$

- $c^T x$: objective function.
- $Ax \leq b$: resource/feasibility constraints.
- $x \geq 0$: nonnegativity constraints.
- A point satisfying every constraint is **feasible**.
- A feasible point with the best objective value is **optimal**.

22. Modeling an LP: The CLRS Political Campaign Example

Let x_1, x_2, x_3, x_4 be thousands of dollars spent on the four campaign issues in CLRS. The campaign must win at least 50k urban, 100k suburban, and 25k rural votes.

$$\begin{array}{ll}\min & x_1 + x_2 + x_3 + x_4 \\ \text{s.t.} & -2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 \\ & 5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \\ & 3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 \\ & x_1, x_2, x_3, x_4 \geq 0.\end{array}$$

Modeling Pattern

1. choose decision variables
2. write constraints
3. write the objective
4. add sign restrictions.

23. Feasible, Infeasible, Unbounded

Feasible

At least one point satisfies every constraint.

Infeasible

No point satisfies all constraints simultaneously.

Unbounded

Feasible solutions exist, but there is no finite optimal objective value.

Important Subtlety

A feasible region may be unbounded even when the LP still has a finite optimal objective value. “Unbounded feasible region” and “unbounded objective” are not identical statements.

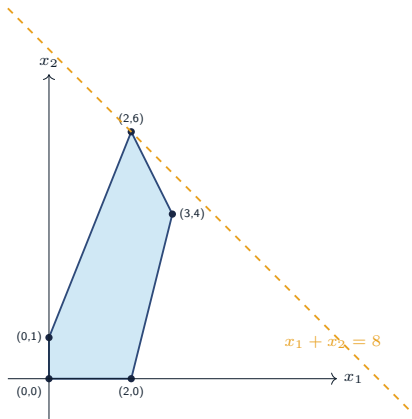
24. Two-Variable LP: The Geometry

Example from CLRS §29.1

$$\begin{array}{ll}\max & x_1 + x_2 \\ \text{s.t.} & 4x_1 - x_2 \leq 8, \\ & 2x_1 + x_2 \leq 10, \\ & -5x_1 + 2x_2 \leq 2, \\ & x_1, x_2 \geq 0.\end{array}$$

- Each inequality gives a half-plane.
- Their intersection is the feasible region.
- The objective $x_1 + x_2 = z$ gives parallel lines.
- Increasing z slides the line in the improving direction.

25. Solving the 2D LP Graphically



Optimal Point

The last objective line that touches the feasible region reaches vertex $(2, 6)$ with value 8.

26. Why an Optimum Occurs at a Vertex

The feasible region is convex: if p and q are feasible, every point on the segment joining them is feasible.

- An objective level set is a line in 2D, a plane in 3D, and a hyperplane in general.
- At the maximum objective value, the level set touches the boundary of the feasible region.
- The contact is either a single vertex or a line segment (in 2D).
- If it is a segment, both endpoints are vertices and are also optimal.

General Statement

Whenever a finite optimum exists, a linear program has an optimal solution at a vertex of its feasible region.

27. Simplex: Geometric Idea

High-Level Algorithm

Start at a vertex and move along an edge to a neighboring vertex whose objective value is no smaller. Stop when no neighboring vertex improves the objective.

- 1 Choose a feasible starting vertex.
- 2 Examine neighboring vertices.
- 3 Move to an improving neighbor.
- 4 Repeat until no improving neighbor exists.

Why termination at a local maximum is enough

Because the feasible region is convex and the objective is linear, the local vertex optimum is a global optimum. CLRS does not develop tableau/pivot arithmetic in this chapter, so we will not either.

28. Simplex: What CLRS Says About Algorithms

Simplex

Widely used in practice, but exponential in the worst case for carefully constructed inputs.

Ellipsoid

Polynomial time, but slow in practice.

Interior-Point

Polynomial time; moves through the interior rather than vertex-to-vertex along the boundary.

Context

These are algorithmic facts from CLRS; the chapter does not derive their full implementations.

Continuous vs Integer LP

General LP is polynomial-time solvable. Requiring all variables to be integers gives integer LP, for which feasibility is NP-hard in general.

29. Duality: Why Do We Need It?

The Question

Suppose x is a feasible solution. How can we certify that no feasible solution has a larger objective value?

- The dual constructs a related minimization LP.
- Every feasible dual solution provides an upper bound on the primal maximum.
- If primal and dual feasible solutions have the *same* objective value, both are optimal.

Analogy

This is the same certification mindset as Max-Flow = Min-Cut: a second object provides a matching bound.

30. Constructing the Dual Mechanically

Primal

$$\max c^T x \quad \text{s.t. } Ax \leq b, x \geq 0.$$

m constraints, n variables.

Dual

$$\min b^T y \quad \text{s.t. } A^T y \geq c, y \geq 0.$$

n constraints, m variables.

Mechanical Recipe

Max $\rightarrow$ Min; RHS b becomes the dual objective coefficients; objective coefficients c become RHS; transpose A ; $\leq$ becomes $\geq$; every primal constraint becomes a dual variable.

31. Worked Dual Example

Primal

$$\begin{aligned} \max \quad & 3x_1 + x_2 + 4x_3 \\ \text{s.t.} \quad & x_1 + x_2 + 3x_3 \leq 30 \\ & 2x_1 + 2x_2 + 5x_3 \leq 24 \\ & 4x_1 + x_2 + 2x_3 \leq 36 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \min \quad & 30y_1 + 24y_2 + 36y_3 \\ \text{s.t.} \quad & y_1 + 2y_2 + 4y_3 \geq 3 \\ & y_1 + 2y_2 + y_3 \geq 1 \\ & 3y_1 + 5y_2 + 2y_3 \geq 4 \\ & y_1, y_2, y_3 \geq 0 \end{aligned}$$

Structure to Memorize

3 primal constraints $\Rightarrow$ 3 dual variables.
3 primal variables $\Rightarrow$ 3 dual constraints.

32. Duality Intuition: Building an Upper Bound

Take nonnegative multipliers y_1, y_2, y_3 of the primal constraints and add them. For the worked example this gives

$$(y_1 + 2y_2 + 4y_3)x_1 + (y_1 + 2y_2 + y_3)x_2 + (3y_1 + 5y_2 + 2y_3)x_3 \leq 30y_1 + 24y_2 + 36y_3.$$

If

$$y_1 + 2y_2 + 4y_3 \geq 3, \quad y_1 + 2y_2 + y_3 \geq 1, \quad 3y_1 + 5y_2 + 2y_3 \geq 4,$$

then, because $x \geq 0$,

$$3x_1 + x_2 + 4x_3 \leq 30y_1 + 24y_2 + 36y_3.$$

Interpretation

A dual feasible point gives a certified upper bound on every feasible primal objective value. The dual minimizes this upper bound.

33. Weak Duality Theorem

Lemma 29.1 — Weak Duality

Let $\bar{x}$ be any feasible primal solution and $\bar{y}$ any feasible dual solution. Then

$$c^T \bar{x} \leq b^T \bar{y}.$$

Proof — Four Essential Steps

$$\begin{aligned} c^T \bar{x} &\leq (A^T \bar{y})^T \bar{x} \\ &= \bar{y}^T A \bar{x} \\ &\leq \bar{y}^T b \\ &= b^T \bar{y}. \end{aligned}$$

dual feasibility and $\bar{x} \geq 0$

rearrange sums / transpose

primal feasibility and $\bar{y} \geq 0$

Exam takeaway

Any feasible dual solution is an upper-bound certificate for the primal maximization problem.

34. Weak Duality as an Optimality Certificate

Suppose you propose $\hat{x}$ for the primal and find a feasible dual solution $\hat{y}$ such that

$$c^T \hat{x} = b^T \hat{y} = K.$$

For every feasible primal x and feasible dual y ,

$$c^T x \leq b^T y.$$

Therefore,

$$c^T x \leq b^T \hat{y} = K = c^T \hat{x}.$$

So no primal solution can beat $\hat{x}$.

Certificate Pattern

feasible primal + feasible dual + equal values $\implies$ both are optimal.

35. Strong Duality

Theorem 29.4 — Strong LP Duality

If the primal and dual are feasible and bounded, then their optimal solutions x^*, y^* satisfy

$$c^T x^* = b^T y^*.$$

- Weak duality gives primal value $\leq$ dual value for every feasible pair.
- Strong duality says the best possible values meet exactly.
- Therefore, a matching primal/dual pair certifies optimality.

Proof depth for this course

CLRS proves strong duality using Farkas' lemma. We will state the theorem and use it, but omit the Farkas/strong-duality proof.

36. Fundamental Theorem of Linear Programming

Theorem 29.5

Every linear program in standard form has one of the following outcomes:

- 1 an optimal solution with a finite objective value;
- 2 no feasible solution (the LP is infeasible);
- 3 feasible solutions exist, but no finite optimum exists (the LP is unbounded).

How the Pieces Fit

Modeling gives the LP; the geometry explains vertices; Simplex searches vertices; duality certifies optimality.

LP: What You Should Know for an Exam

- Write an LP from a word problem: variables, objective, constraints, nonnegativity.
- Explain feasible, infeasible, unbounded, and optimal solutions.
- Solve a 2-variable LP geometrically and identify the optimal vertex.
- Explain why a finite optimum occurs at a vertex.
- Describe the Simplex idea as vertex-to-vertex improvement.
- Construct the dual mechanically and keep track of the dimensions.
- Explain the upper-bound interpretation of the dual.
- State and prove Weak Duality (short algebraic proof).
- State Strong Duality and use equality of primal/dual values as an optimality certificate.

Not required here

Simplex tableau/pivot arithmetic and the proof of Farkas' lemma / Strong Duality.

Final Wrap-Up: One Common Algorithmic Idea

Reformulate $\longrightarrow$ Make the Hard Part Easy

- **FFT:** coefficient form $\rightarrow$ point-value form makes polynomial multiplication pointwise and linear-time.
- **LP duality:** primal $\rightarrow$ dual turns optimality into a pair of matching bounds.

Core Results to Remember

FFT: $\Theta(n \log n)$

Polynomial multiplication: $\Theta(n \log n)$

Weak Duality: $c^T x \leq b^T y$